

Field Audit Intelligence — Written Summary

BroadPeak AI Engineer Take-Home · Demo location: Wolf Creek Golf Club, Atlanta, GA

Approach

I built a four-agent pipeline (Pydantic AI + Groq) that turns raw field-audit input into evidence-grounded findings, correlates them with recent public reviews, and drafts a franchisee-facing corrective action plan:

Clarification → Findings → Review Correlation → Corrective Action.

The clarification step is the core anti-hallucination guardrail: before any finding is written, a fast model checks whether the consultant's checklist responses and notes are specific enough to support one. Vague input ("floor looked a little dirty") gets bounced back with targeted follow-up questions instead of being turned into a guessed finding. This directly answers the brief's central question — *how do you prevent the system from asserting a violation the input doesn't support* — with a design answer, not just a prompt instruction: the pipeline structurally cannot reach the findings step on ambiguous input without either an answer or an explicit "proceed anyway" override, which then forces `requires_human_review = True` on the affected findings rather than trusting the model's inference.

Interaction model: a structured checklist form is the primary interface, because the primary user *in the moment* is a field consultant who needs speed, not a chatbot. The system drops into a short conversational follow-up only when it genuinely can't proceed — form speed for the common case, conversational precision for the ambiguous 20%.

Two audiences, one dataset: the same audit report renders two ways — a Consultant View (confidence scores, human-review flags, raw evidence, full prompt/response audit trail) and a Franchisee View (clean summary, respectful cover message, action table with deadlines, no internal scoring). This reflects that the consultant needs to *defend* a finding and the franchisee needs to *act* on it — different jobs, different UI.

Review correlation is explicitly hedged. Recent negative reviews are summarized into themes and only linked to audit categories with correlation language ("may relate to," never "confirms" or "proves"), with a mandatory caveat about small sample size baked into the schema itself — not left to prompt discipline alone.

Scaling across brands: rather than one generic franchise checklist, I tailored the checklist categories and RAG-backed brand standards to what a golf & hospitality property actually looks like (course conditions, pace of play, halfway-house food safety) instead of forcing a retail-style audit onto Wolf Creek. Brand standards live in ChromaDB keyed by (brand, category); the same agent code onboards a new BroadPeak brand by adding new standards documents, not new code.

Key trade-offs and why

- **Model tiering** (Groq `llama-3.1-8b-instant` for classification-style steps, `llama-3.3-70b-versatile` for judgment-heavy findings/corrective actions) balances cost and quality — relevant once this runs several LLM calls across hundreds of locations per audit.
- **Groq over a frontier model** for this POC specifically to keep a public live demo fast and effectively free to run; the provider is isolated to one function (`backend/agents.py::_model`), so swapping providers later is a small, contained change.

- **A dependency-free hashing embedding for ChromaDB** instead of a downloaded sentence-transformer model — removes a fragile runtime network dependency from container cold-start on a public demo, at the cost of weaker semantic fallback matching (mitigated by an exact-match-first retrieval strategy, since checklist categories are authored to match standards categories 1:1 in this POC).
- **SQLite over Postgres, and a Places-API mock fallback** — both keep the POC runnable by anyone with just a Groq key, at the obvious cost of not being multi-tenant or requiring live Google credentials to see the full flow end-to-end.
- **The Places API's review-history cap** (a handful of recent reviews per call, no way to page further back) is handled by upserting every fetch into a local snapshot table keyed by a stable hash, so repeated audits over time accumulate a longer effective history than any single call returns — without a higher-tier review product.

Results

The pipeline runs end-to-end: checklist + notes → clarification gate → structured findings with cited evidence and severity → review theme correlation → franchisee corrective action plan, all persisted with a full prompt/response audit trail per step for explainability. I tested the live system with real Groq calls on both designed paths: specific notes sail through the pre-check to a completed report (findings cite verbatim evidence and the retrieved brand standard; deadlines scale with severity), while thin input is bounced to targeted clarifying questions and — if left unanswered under "proceed anyway" — comes back flagged for human review rather than asserted. The Franchisee View exposes none of the internal scoring, and review correlation surfaces hedged themes with the sample-size caveat visible. The same build runs locally and as a Dockerized Hugging Face Space demo with a single secret set.

Next steps before this touches a real franchise location

1. **Human-in-the-loop review UI** for anything flagged `requires_human_review`, with sign-off tracked in the audit trail, before a report reaches a franchisee.
2. **Real photo upload + vision-model analysis**, replacing the text stand-in used in this POC.
3. **A governed standards-authoring workflow** (versioning, approval) so franchise-ops teams can maintain brand standards without an engineer touching seed data.
4. **Postgres + migrations** for multi-tenant durability, replacing SQLite.
5. **Cost/rate monitoring and an evaluation harness** (golden audits) to catch prompt regressions before hundreds of locations are calling this several times per visit, plus a stronger embedding model for RAG once the network-dependency trade-off above is worth revisiting.